

Bedrock Domain Scenario



Bedrock Domain Scenario

CONFIDENTIAL - FOR INTERNAL USE BY BADGE CANDIDATES ONLY

Apex & Visualforce

Instructions

Presentation Time : 35 minutes

Q&A : 20 minutes

Immediate Feedback : 5 minutes

For the hypothetical scenario described below, you will need to document and present your proposed solution design followed by a Q&A session with a judge. As part of the solution you should:

- Describe an appropriate solution that meets the business requirements
- Deliver all of the required deliverables as part of that presentation
- Address trade-offs and considerations (scale, performance etc.) for your solution options

You should target the presentation to an audience of executives that have substantial technical experience but little Salesforce hands-on experience. In cases where requirements are not explicitly stated for the scenario, you should use your best judgment and make appropriate assumptions based on the information provided and that in the [Bedrock Background document](#). You should indicate any assumptions that were made when designing the proposed solution. You will not have an opportunity to ask clarifying questions related to the hypothetical scenario.

You will be evaluated on your ability to assess the scenario requirements, design a solution, communicate the proposed architecture, and justify the design decisions.

The executive will evaluate your solution in the following areas:

Objective	Weighting
Able to identify business requirements and understand the needs and challenges of a scenario	5%
Able to leverage the features and capabilities of the platform appropriately within the solution	10%
Able to clearly articulate the solution design	10%
Able to recommend the appropriate Apex execution context based on the Use Case,	10%

whether synch, async or batch	
Has an understanding of Object Oriented Programming concepts and how they apply to Apex	5%
Has an understanding of Apex governor limits and how to write efficient code	5%
Understands when to use Apex vs declarative configuration. Has an understanding of when to recommend the usage of a Trigger and able to recommend the appropriate DML events (Before, After, Insert, Update, Delete, Undelete)	10%
Has an understanding of Batch processing capability and able to recommend appropriate usage based on the use case	5%
Has an understanding of the differences between Lightning Web Components, Aura, and Visualforce and able to make a recommendation based on the use case	10%
Has an understanding of building Lightning Web Components and integrating them into Lightning Experience.	10%
Has an understanding of LWC, and how it can be used to override standard user interfaces	5%
Able to design a scalable solution	5%
Has an understanding of Salesforce Data Cloud and AI capabilities, and is able to implement Retrieval-Augmented Generation (RAG) solutions using Apex and integrate intelligent insights into Lightning Web Components.	10%

Expected Deliverables

- Please use the attached template to guide your presentation. You may add slides if required to further explain your solution.
- Present a functioning demo of the core components of your solution where specifically called for in the sections.

Problem Overview

Bedrock utilizes the Lightning interface and has developers on staff. Their Center of Excellence has mandated that Salesforce's latest development methods and best practices are used. Speed to answers is extremely important for advisors when they are on phone calls with clients. Real-time quotes, account summaries, and account plans (PDFs) are essential to client success. Nightly batches need to be run to ensure that data is updated with the latest quotes (integration not required, but needs to be alluded to).

Section 1

Bedrock would like to add a component to the sales app home page that shows a map with a pin marking Bedrock's corporate headquarters. The address is 'The Landmark @ One Market, Suite 300', San Francisco, CA 94105. Ensure that the map is zoomed in to show as much detail as possible while still being able to see the city name. Utilize modern and efficient techniques to create the component in an IDE and deploy to an org, to demonstrate this map on the home page. Be prepared to demo in the org and discuss your code in the IDE. Consider, but do not implement, general LWC testing and security concepts.

Section 2

Bedrock Enterprises uses a set of custom pages to display a highly stylized and formatted Client Plan for each customer. This Client Plan displays key customer data points (pulled in from the Account & main Contact records), summaries of their opportunity pipeline and their service entitlements.

The Bedrock sales team has had access to server-side VF-to-PDF functionality which was used to reliably transform the text-only Client Plan page into a PDF that would get stored against the Customer Account record as a related File. They do this so they can easily print it out before their quarterly meetings with each customer and use it to drive the conversation.

Bedrock developers are about to add an interactive JavaScript map to the Client Plan Lightning Record page at the request of the CIO. This map is used to dynamically highlight all countries where Bedrock is doing business with this particular customer as well as highlighting key countries that are seen as growth markets.

This new map will also be included in the Client Plan PDF in such a way that sales users do not need to use print-screen or save-as functionality from the browser. The Client Plan should get generated, and attached to the Account record as a File, through a single button click on the Client Plan page.

Describe the challenges Bedrock Enterprises developers might face to achieve this and propose a solution. Code is not required.

Section 3

Bedrock Enterprises has a back-end system that stores and aggregates quotes given out to customers and prospects from across their Healthcare offices. Each quote consists of the main quote record and up to 10 child records containing quote details. The requirement is for this data, that is not yet in Salesforce, to be stored as standard Quote object data against the Salesforce Account records through a Batch process.

Assume that there is no ETL tool with advanced capabilities available within Bedrock Enterprises. Also assume there are two custom objects within Salesforce that act as a staging area, `Healthcare_Quote_Staging__c` and `Healthcare_QuoteDetail_Staging__c`. Final assumption; an integration is in place that will, via an automated process, start the following actions at 11 pm every evening:

- Clear both staging objects in Salesforce
- Populate both staging objects with new quotes and quote records that have been updated in the back-end system since the last run of this integration process

With a 'dumb' middleware in place, the processing responsibilities of this staged Quote data falls on Salesforce. Design a native solution that will process all staged data and that can either create or update standard Quote records and associate these with the customer Account records. The volume of records in `Healthcare_Quote_Staging__c` can be up to 1 Million for each run.

Consider, amongst others, elements such as process initiation, logic encapsulation, error handling and testing. Make reasonable assumptions where necessary. Write pseudo code of the solution.

Section 4

Bedrock Enterprises record Contacts which are related to multiple Accounts, so that they can easily see the relationships between people and businesses. They would like the users to see the indirect Account relationships via Contacts directly on the Account record using a custom Lightning Web Component for direct & indirect accounts eg (Account name, industry, number of contacts, Life time value....etc . The admins would be able to drop this component interactively on the record detail page using the App Builder. The admins should also be able to configure it so that the number of items shown can be changed and the list of items can be sorted by name or number of times the relationship occurs. Be prepared to show and discuss your code.

Bedrock Enterprise has a unique business process, defined by regional Sales Managers for that region based on the attributes of deal and the customer (Customer Rating, Amount, Region etc...). These attributes are dynamic & should be

configured by the Sales Managers. The Admins would be able to drop this custom lightning component on any Opportunity page using App Builder. The component should render as chevron or buttons, each item click on the chevron opens a modal dialog with a specific process like view, create and update related records (customer, orders, tasks, events, view hierarchy, view documents, etc.,) configured by the sales manager. Support, HR & Marketing team also has similar processes, would like to leverage the same component on other objects. Be prepared to demonstrate your solution and show the code.

Being part of highly regulated industries, it is important to the Security Office that this component is safe and does not inadvertently make information available to other third-party components they may choose to add from the AppExchange in future; unless the components are explicitly authorized to access it. The look and feel of this component should be in keeping with the standard Lightning Experience UI.

Section 5 - Other requirements

Orders - When an Opportunity is Closed Won, an Order record should be automatically generated, and the Opportunity Line Items should be synced across as line items of the order. A flag 'Ready for Integration' should be set on the order, and an event notification should be triggered to the ESB so that this Order may be pushed to the Sales Order Processing system. Bedrock may have thousands of line items per Opportunity and requires a solution that will scale. Code is not required.

Extensibility - Bedrock has had difficulty in the past with business logic being spread across multiple classes, and other logic-based tools, with no way of knowing in what order the logic is executed. They would like a recommendation for the appropriate way to consolidate and coordinate event-driven logic in Salesforce. Recommendations should include both declarative and programmatic options.

Plagued by an inflexible 'green screen' system, which requires a lot of changes for enablement and disablement of rules, Bedrock wants the system that has any large chunks of customization to be enabled or disabled based on factors such as the user or user's profile. It should also provide a manual override for cases when they might need to load in large volumes of data, as they may need to suspend some of the custom business logic whilst the data load is in progress and then apply the business logic after the data has been loaded in. Code is not required."

Opportunity - When a new Opportunity is created the business requirement is that the name of the Opportunity is pre-populated with the current Date + Time on the standard Opportunity page (even before hitting Save). This should work both when the New Opportunity button is used from the Account-page related list as well as from the New Opportunity button on the Opportunity Tab. Code is not required.

Section 6 – Data Centricity

Bedrock Enterprises is now focused on building a unified 360° customer profile by leveraging Salesforce's Data Cloud to connect data from multiple disparate systems. This initiative aims to harmonize customer information from sources such as CRM (Salesforce Accounts/Contacts), the back-end quote system, service/support databases, and even marketing or ERP systems into a single Customer 360 profile.

The challenge for Bedrock developers is to design a solution that can ingest and consolidate high volumes of data (potentially millions of records) into Data Cloud's unified profile store, using identity resolution rules to merge duplicates and link records for the same customer across systems. They must ensure that data mapping into the Customer 360 Data Model is consistent and handle conflicts (for example, if different systems have different values for the same customer attribute).

The profile data should be kept up-to-date in near real-time or via efficient nightly batches – for instance, scheduling data ingestions or using connectors at 11pm each night to pull in new or changed records since the last run. An additional consideration is how this unified profile will surface in the Sales Cloud UI for advisors: the team may need to create a custom Lightning component or use Data Cloud's native connectors so that when a user views an Account or Contact in Salesforce, they can quickly see the aggregated profile details from Data Cloud. Modern best practices would be applied – using an API or connector approach rather than manual CSV imports, encapsulating the integration logic in a robust service with proper error handling and retry mechanisms. The developers should also plan for scalability and performance: Data Cloud is built for large-scale data, but any pushing data back to core Salesforce must be done in bulk-safe ways .

Testing would include verifying that identity resolution correctly merges records and that the unified profile updates do not hit governance limits in Salesforce.

Section 7 – AI Initiatives

Bedrock Enterprises wants to leverage AI capabilities on top of this unified data. They are exploring an AI-driven assistant that can help advisors get instant answers and insights during client calls, using the comprehensive data now available. The vision is to implement a solution where a Generative AI can answer complex questions or provide summaries about a customer by referencing the 360° profile and other knowledge, using a Retrieval Augmented Generation (RAG) approach.

In practice, Bedrock's developers might build a custom component where an advisor could ask natural language questions like "What products has this client purchased in the last year and what is their support history?" or "Summarize this customer's overall engagement with us for this quarter." Behind the scenes, the component retrieve relevant data – for instance, querying the unified profile in Data Cloud for recent transactions, pulling related cases from Salesforce Service Cloud, and perhaps

retrieving pertinent knowledge articles or PDF account plans – and then call an AI prompts to generate a coherent answer.

Implementing RAG through Apex means the team must handle the steps of fetching data and constructing a prompt for the AI.

This could involve using Salesforce's Einstein GPT . and send it along with the advisor's question to the AI model, which would return a narrative answer or recommendation. The developers need to address several challenges here: ensuring the prompt stays within size limits and focuses on relevant data and preventing inaccuracies or "hallucinations" by the AI. They might use prompt templates and few-shot examples to guide the model's style and correctness, and validate the answers before displaying. The AI component would be configured so that admins can deploy it on record pages via App Builder, and it should allow some configuration.

Bedrock Enterprises has deployed a Screen Flow on the Account page to let advisors pose natural-language questions about past customer meetings. The flow performs a hybrid search across the custom Meeting_Minutes__c object, then passes those results into an AI prompt to generate a concise summary.

However, several challenges remain unsolved:

- **Dynamic Language Requirements:** Prompt Templates invoked directly from Flow can't adapt the response language based on the user's locale or account settings.
- **Control & Visibility:** Without Apex, there's no mechanism to inject custom logging, robust error handling, or input pre-processing to improve prompt accuracy and traceability.
- **User Experience Risks:** Inconsistent language or unhandled errors can lead to advisor confusion, hamper adoption, and expose incomplete or misleading summaries.

Bedrock needs a solution that preserves the declarative simplicity of Screen Flows while providing the necessary language controls, operational visibility, and reliability in AI-driven summarization.